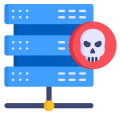


Bangladesh GPCA Malware

Initial Context:

- **The Bangladesh (central) bank heist, happened in 2016**
- **The attackers tried to transfer funds with SWIFT transactions**
- **Artifacts: nroff b.exe (1d0e79feb6d7ed23eb1bf7f257ce4fee) and gpca.dat**
- **We need to decrypt gpca.dat, that is known to be used by nroffb.exe**

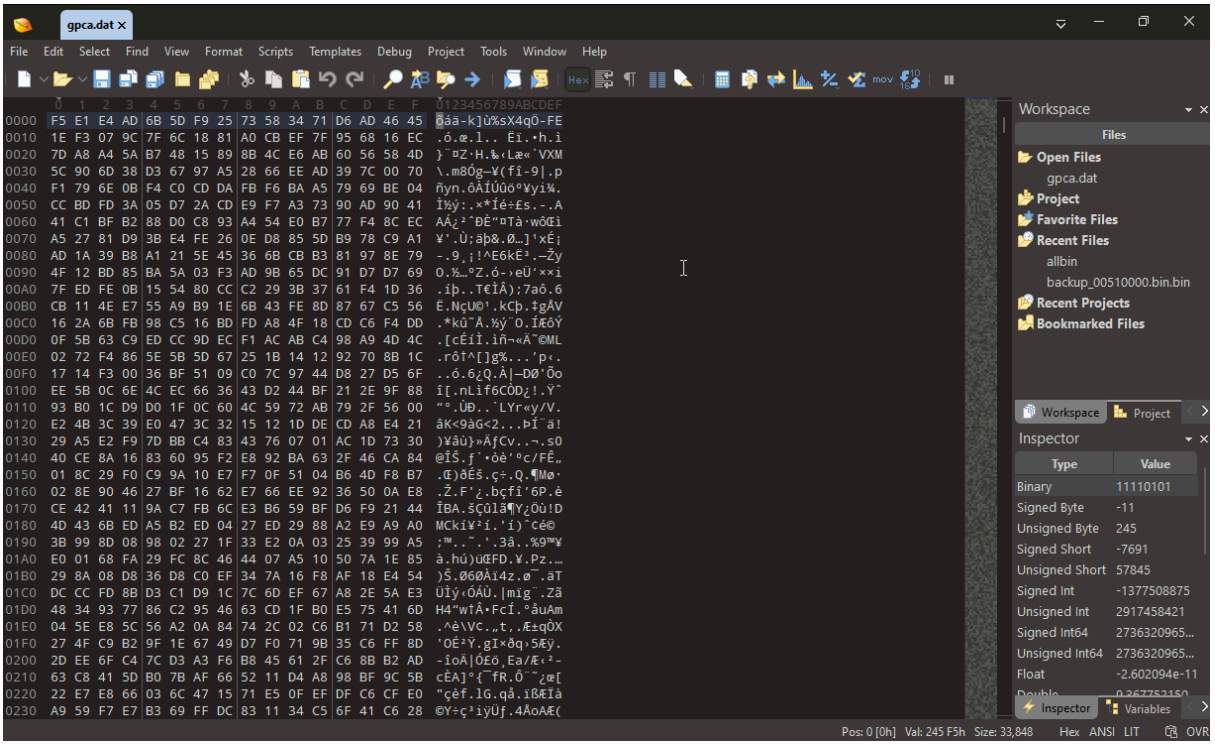
The executable reads .dat file that is encrypted and it is supposedly some configurations.

that we do need to recover and decrypt.

File: **gpca.dat**

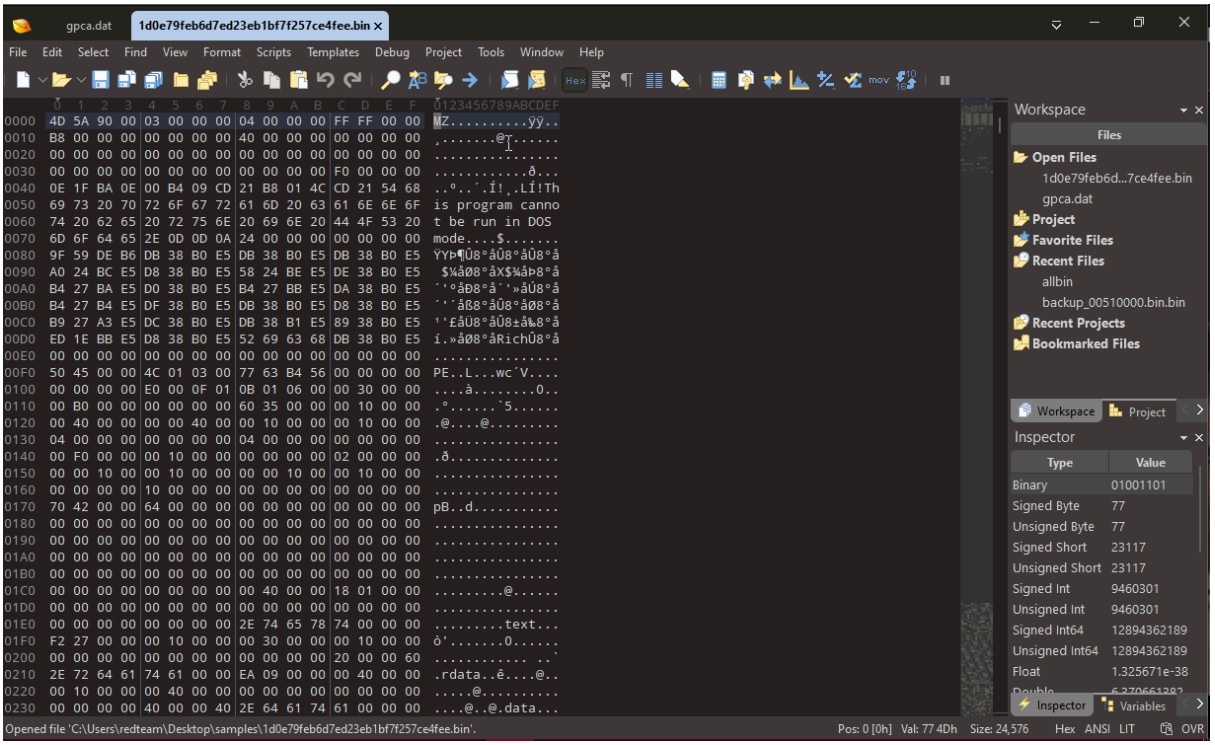
Toool: 010 editor

This looks like heavily encrypted.



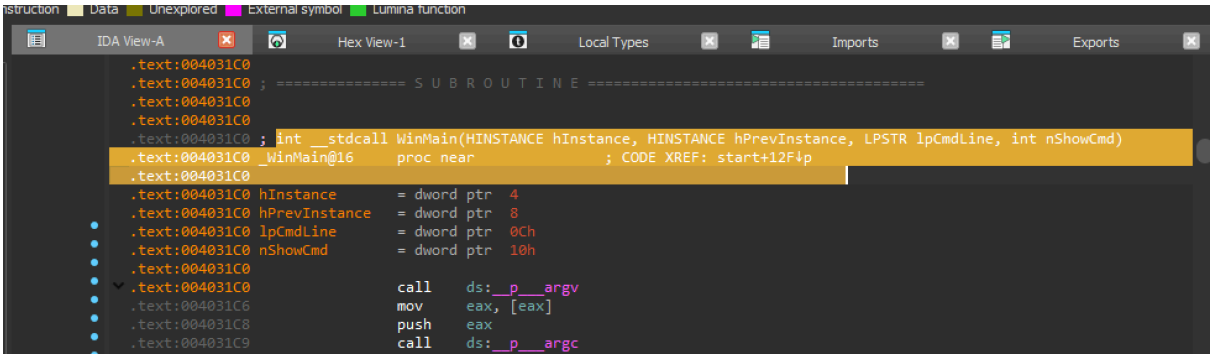
File: 1d0e79feb6d7ed23eb1bf7f257ce4fee.

This is a exe file we have MZ header, DOS stun and we have the PE magic.

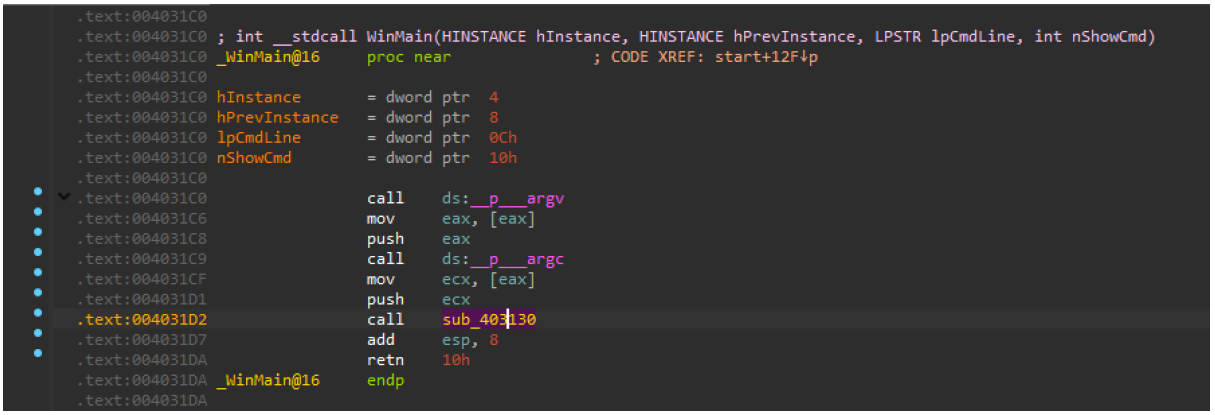


IDA CODE Analysis

Start with main. IDA recognize that it is a C compiled binary.



Nothing interesting let's follow in the call to `function 0x403130`



looks like code nothing that requires immediate attention.

Our goal: Decrypts the content of the file called gpca.dat

Bangladesh GPCA - Tracking the value

Data flow: tracking values at various points in the program "gpca.dat" - obvious string value to look for

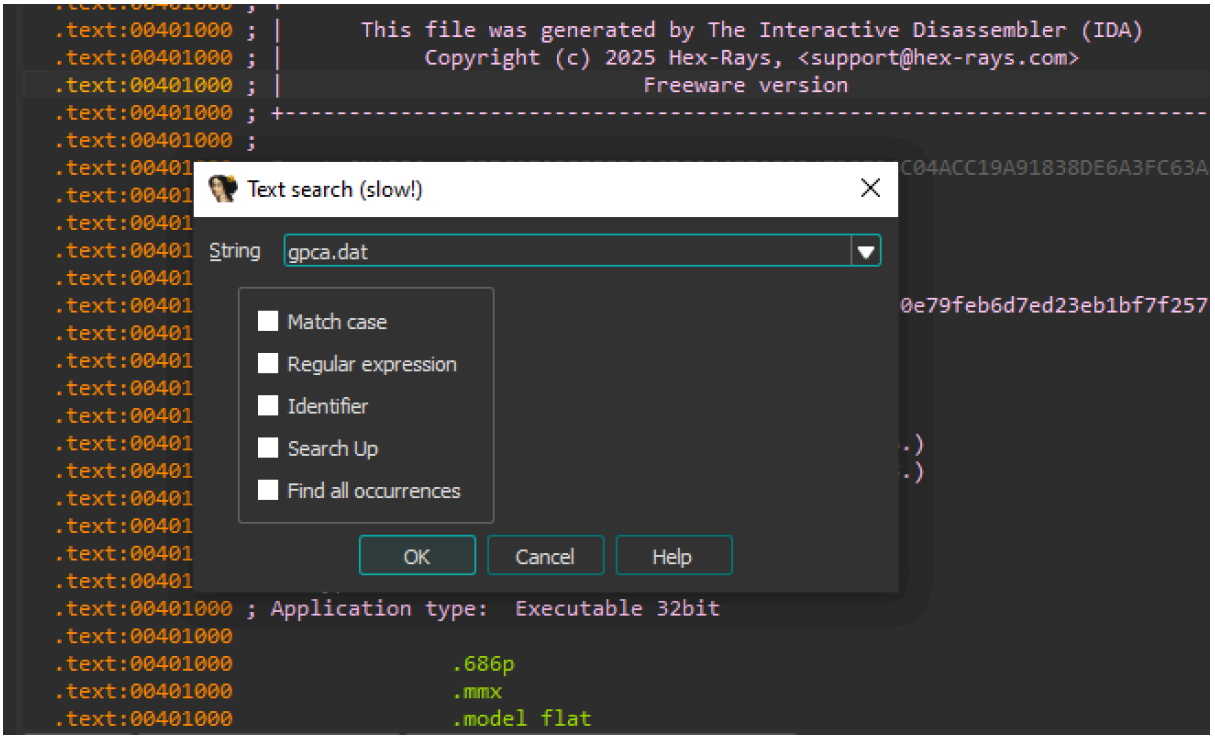
- read/write/offset cross-references
- "what reads this value?"
- "What writes this value?"
- "what uses the address of this value?"

Code (control) flow: determining possible sequences of execution

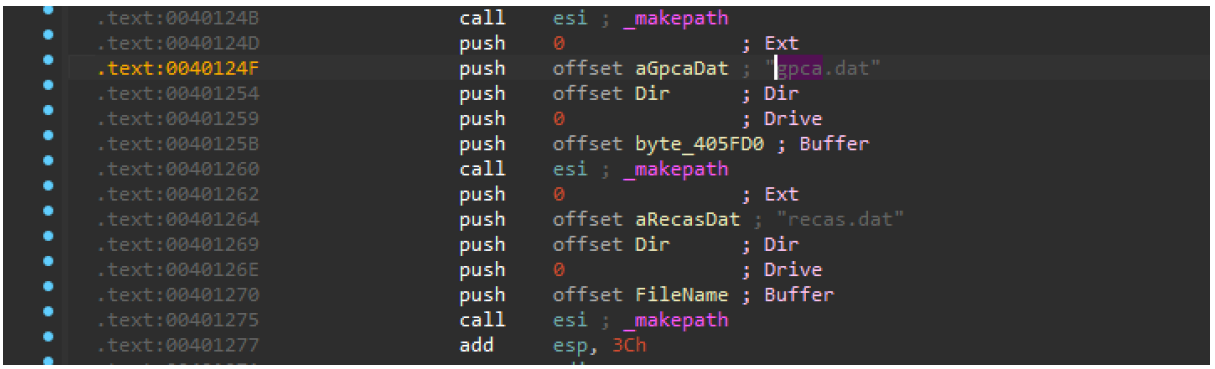
- call/jump/offset cross-references
- "what calls this code?"
- "What jumps to this code?"
- "what uses the address of this code?"

Our starting point is finding a strings containing "gpca.dat" and then further to any string that was generated by using this string. By using all these references we find the code that is using this file name.

Search string in IDA



1st - Found “gpca.dat” this is an argument to a function.



Navigate to the location of the string. Look at the cross-references.

```
.data:0040503C ; char aRecasDat[]
.data:0040503C aRecasDat        db 'recas.dat',0        ; DATA XREF: sub_4010D0+194↑o
.data:00405046                  align 4
.data:00405048 ; char aGpcaDat[]
.data:00405048 aGpcaDat         db 'gpca.dat',0         ; DATA XREF: sub_4010D0+17F↑o
.data:00405051                  align 4
.data:00405054 ; char aMcm[]
.data:00405054 aMcm             db 'mcm',0              ; DATA XREF: sub_4010D0+13D↑o
.data:00405058 ; char aFoff[]
.data:00405058 aFoff            db 'foff',0             ; DATA XREF: sub_4010D0+128↑o
.data:0040505D                  align 10h
.data:00405060 ; char aFofp[]
.data:00405060 aFofp            db 'fofp',0             ; DATA XREF: sub_4010D0+113↑o
.data:00405065                  align 4
.data:00405068 ; char aNfzf[]
.data:00405068 aNfzf            db 'nfzf',0             ; DATA XREF: sub_4010D0+FB↑o
.data:0040506D                  align 10h
.data:00405070 ; char aNfzp[]
.data:00405070 aNfzp            db 'nfzp',0             ; DATA XREF: sub_4010D0+E6↑o
.data:00405075                  align 4
.data:00405078 ; char aMcs[]
.data:00405078 aMcs             db 'mcs',0              ; DATA XREF: sub_4010D0+D1↑o
.data:0040507C ; char aUnk[4]
.data:0040507C aUnk             db 'unk',0              ; DATA XREF: sub_4010D0+BC↑o
.data:00405080 ; char aOut[4]
.data:00405080 aOut             db 'out',0              ; DATA XREF: sub_4010D0+A4↑o
.data:00405080                                          ; sub_4010D0+16A↑o
.data:00405084 ; char aIn[3]
```

2nd Time we find the same string.

```
.data:00405048 ; char aGpcaDat[]
.data:00405048 aGpcaDat         db 'gpca.dat',0         ; DATA XREF: sub_4010D0+17F↑o
.data:00405051                  align 4
.data:00405054 ; char aMcm[]
.data:00405054 aMcm             db 'mcm',0              ; DATA XREF: sub_4010D0+13D↑o
```

So this is the only location in the whole binary. That mentions our file name.

```
.text:0040123A                  push    offset aOut     ; "out"
.text:0040123F                  push    offset byte_405CC4 ; Dir
.text:00401244                  push    0               ; Drive
.text:00401246                  push    offset byte_405DC8 ; Buffer
.text:0040124B                  call    esi ; _makepath
.text:0040124D                  push    0               ; Ext
.text:0040124F                  push    offset aGpcaDat ; "gpca.dat"
.text:00401254                  push    offset Dir      ; Dir
.text:00401259                  push    0               ; Drive
.text:0040125B                  push    offset byte_405FD0 ; Buffer
.text:00401260                  call    esi ; _makepath
.text:00401262                  push    0               ; Ext
.text:00401264                  push    offset aRecasDat ; "recas.dat"
.text:00401269                  push    offset Dir      ; Dir
.text:0040126E                  push    0               ; Drive
```

Analysing

We see that it is used as an argument to the function called _makepath.

we don’t know what is the meaning of the arguments. we will us ida’s definition of the function.

So, in our case, gpca.dat is the argument Filename.

MSDN Refrences.

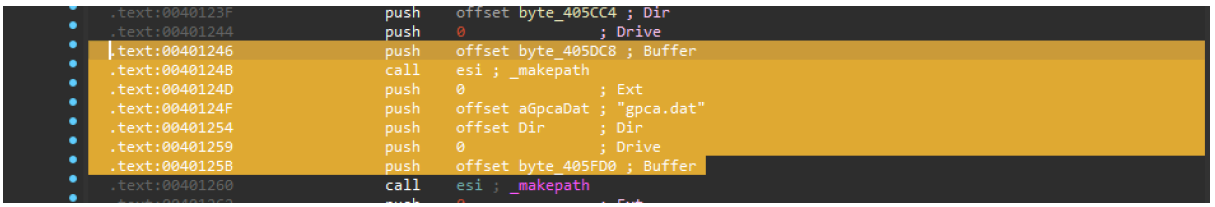
`_makepath`, `_wmakepath`

Create a path name from components. More secure versions of these functions are available; see _makepath_s, _wmakepath_s.

Syntax

C Copy

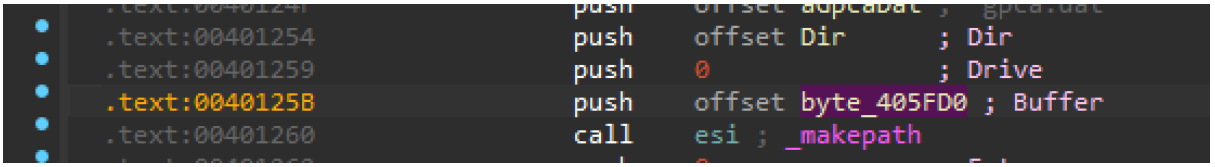
```
void _makepath(
   char *path,
   const char *drive,
   const char *dir,
   const char *fname,
   const char *ext
);
void _wmakepath(
   wchar_t *path,
   const wchar_t *drive,
   const wchar_t *dir,
   const wchar_t *fname,
   const wchar_t *ext
);
```



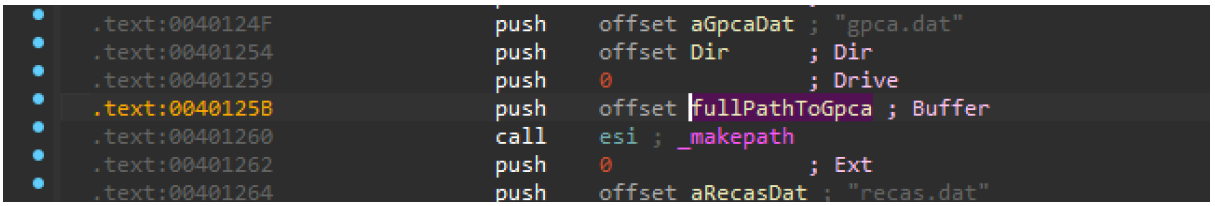
Our file name is used as fname here.

So this function will compile a full path to a file name from it’s components. and store it in the buffer pointed by the first argument.

In our case this buffer was important to us because it will also be used as a longer path to the same file name.

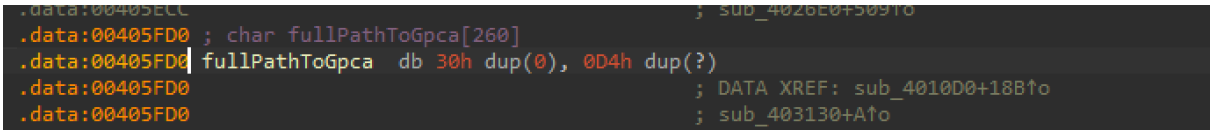


let's rename this buffer to for us to `fullPathToGpca` trace it's value. this new fullpath is what we are going to trace for both code and data analysis.



---

follow the `fullPathToGpca` .

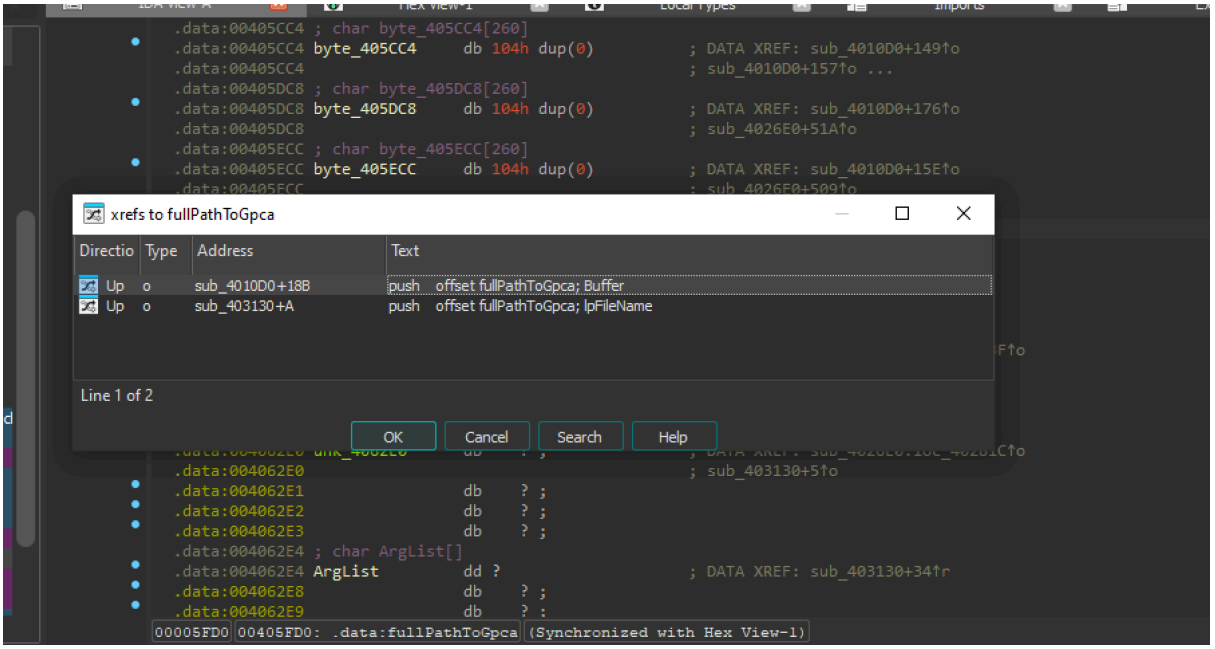


Now let's use this fullPath Buffer at `0x00405FD0`

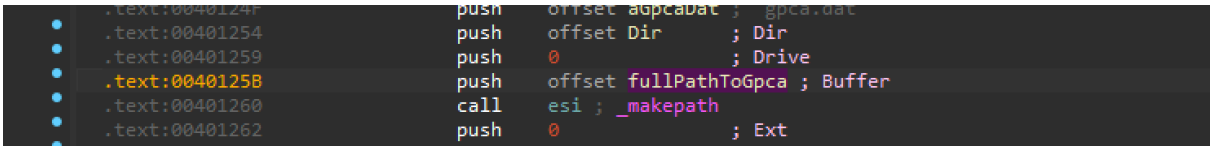
We will use back references generated by IDA to track the places in the code where this value is used.

Press x to see back references. There are 2 places in the code that references this address.

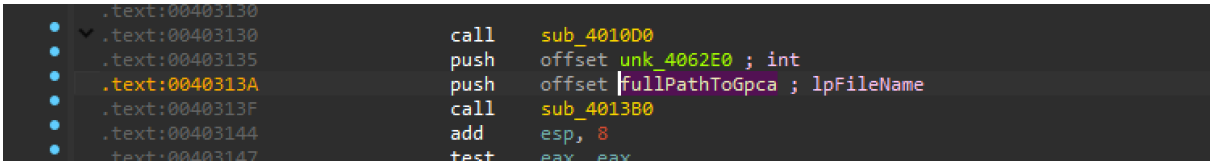
inspect code at both the addresses.



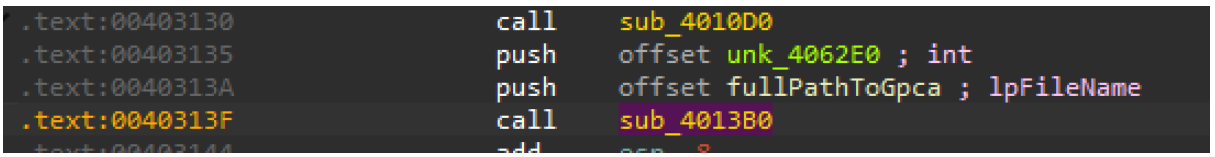
The 1st location we have seen already.



The 2nd one is at `0x0040313A` The address of the buffer is passed on the stack.



look's like it's used in this argument to another function.



Into the function `0x4013B0`

```
.text:004013B0 var_4           = dword ptr -4
.text:004013B0 lpFileName      = dword ptr  4
.text:004013B0 arg_4           = dword ptr  8
.text:004013B0
.text:004013B0                 push    ecx
.text:004013B1                 mov     ecx, [esp+4+lpFileName]
.text:004013B5                 push    ebx
.text:004013B6                 lea     eax, [esp+8+var_4]
.text:004013BA                 push    esi
.text:004013BB                 push    eax             ; int
.text:004013BC                 push    ecx             ; lpFileName
.text:004013BD                 or      esi, 0FFFFFFFFh
.text:004013C0                 mov     [esp+14h+var_4], 0
.text:004013C8                 call    sub_401290
.text:004013CD                 mov     ebx, eax
.text:004013CF                 add     esp, 8
.text:004013D2                 test    ebx, ebx
.text:004013D4                 jnz     short loc_4013DD
.text:004013D6                 pop     esi
.text:004013D7                 or      eax, 0FFFFFFFFh
.text:004013DA                 pop     ebx
.text:004013DB                 pop     ecx
.text:004013DC                 retn
.text:004013DD ; ---------------------------------------------------------------------------
```

It copies to ecx then again passed as an argument function `0x401290`

```
.text:004013BA                 push    esi
.text:004013BB                 push    eax             ; int
.text:004013BC                 push    ecx             ; lpFileName
.text:004013BD                 or      esi, 0FFFFFFFFh
.text:004013C0                 mov     [esp+14h+var_4], 0
.text:004013C8                 call    sub_401290
.text:004013CD                 mov     ebx, eax
.text:004013CF                 add     esp, 8
.text:004013D2                 test    ebx, ebx
.text:004013D4                 jnz     short loc_4013DD
.text:004013D6                 pop     esi
.text:004013D7                 or      eax, 0FFFFFFFFh
```

In `0x401290`

Again it was renamed to lpFileName and copied to eax. and it is passed to function `CreateFileA`.

```
.text:00401290
.text:00401290 lpFileName      = dword ptr  4
.text:00401290 arg_4           = dword ptr  8
.text:00401290
.text:00401290                 mov     eax, [esp+lpFileName]
.text:00401294                 push    ebx
.text:00401295                 push    0               ; hTemplateFile
.text:00401297                 push    80h             ; dwFlagsAndAttributes
.text:0040129C                 push    3               ; dwCreationDisposition
.text:0040129E                 push    0               ; lpSecurityAttributes
.text:004012A0                 push    0               ; dwShareMode
.text:004012A2                 push    80000000h       ; dwDesiredAccess
.text:004012A7                 push    eax             ; lpFileName
.text:004012A8                 call    ds:CreateFileA
.text:004012AE                 mov     ebx, eax
.text:004012B0                 cmp     ebx, 0FFFFFFFFh
.text:004012B3                 jnz     short loc_4012B9
.text:004012B5                 xor     eax, eax
.text:004012B7                 pop     ebx
.text:004012B8                 retn
```

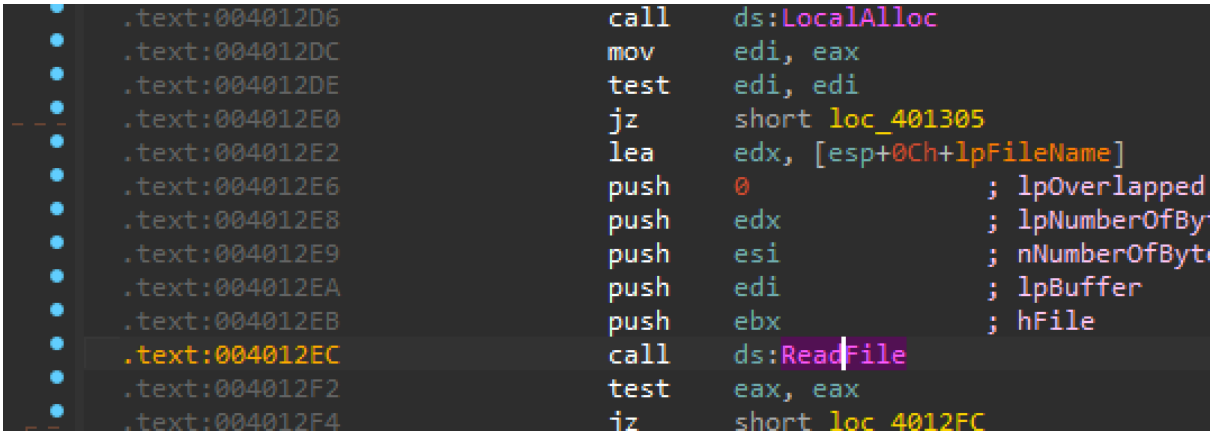
We can see that the opened file handle is used to get the file size.

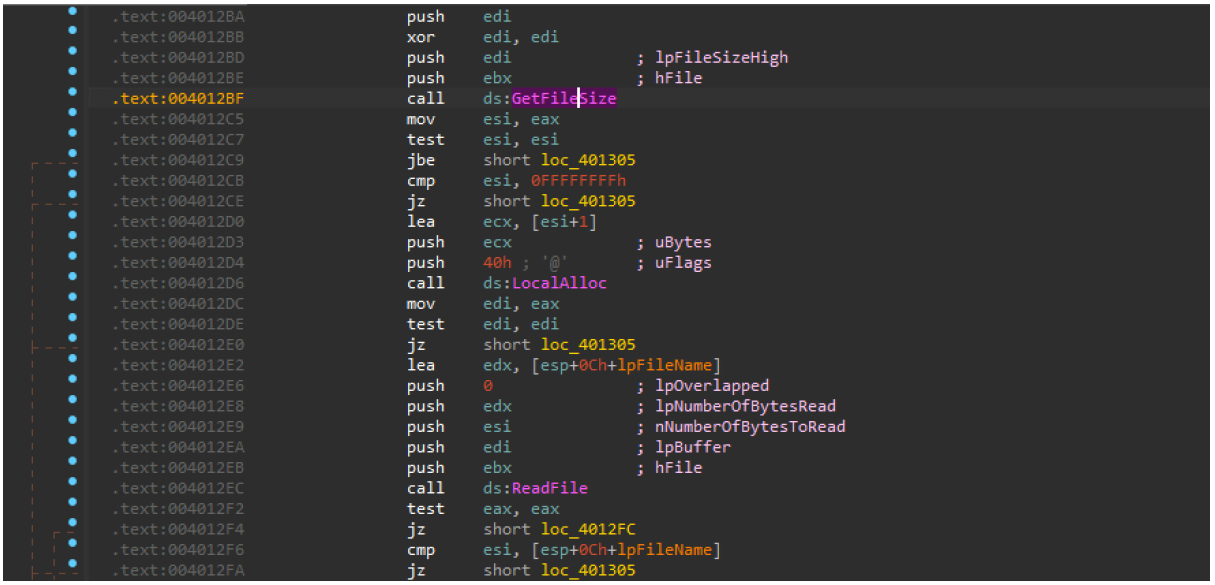
```
.text:004012AE                 mov     ebx, eax
.text:004012B0                 cmp     ebx, 0FFFFFFFFh
.text:004012B3                 jnz     short loc_4012B9
.text:004012B5                 xor     eax, eax
.text:004012B7                 pop     ebx
.text:004012B8                 retn
.text:004012B9 ; ---------------------------------------------------------------------------
.text:004012B9
.text:004012B9 loc_4012B9:                             ; CODE XREF: sub_401290+23↑j
.text:004012B9                 push    esi
.text:004012BA                 push    edi
.text:004012BB                 xor     edi, edi
.text:004012BD                 push    edi             ; lpFileSizeHigh
.text:004012BE                 push    ebx             ; hFile
.text:004012BF                 call    ds:GetFileSize
.text:004012C5                 mov     esi, eax
.text:004012C7                 test    esi, esi
.text:004012C9                 jbe     short loc_401305
.text:004012CB                 cmp     esi, 0FFFFFFFFh
.text:004012CE                 jz      short loc_401305
.text:004012D0                 lea     ecx, [esi+1]
.text:004012D3                 push    ecx             ; uBytes
.text:004012D4                 push    40h ; '@'       ; uFlags
.text:004012D6                 call    ds:LocalAlloc
.text:004012DC                 mov     edi, eax
.text:004012DE                 test    edi, edi
.text:004012E0                 jz      short loc_401305
```

Then there is a Memory Allocation. for that file size+1

```
.text:004012C7                 test    esi, esi
.text:004012C9                 jbe     short loc_401305
.text:004012CB                 cmp     esi, 0FFFFFFFFh
.text:004012CE                 jz      short loc_401305
.text:004012D0                 lea     ecx, [esi+1]
.text:004012D3                 push    ecx             ; uBytes
.text:004012D4                 push    40h ; '@'       ; uFlags
.text:004012D6                 call    ds:LocalAlloc
```

Then the file is read. And the buffer is the result of the memory allocation. The number of byte read is the file size.





So the whole file is read into a newly allocated buffer.

and if something goes wrong the buffer is free.

```
.text:004012EB                 push    ebx             ; hFile
.text:004012EC                 call    ds:ReadFile
.text:004012F2                 test    eax, eax
.text:004012F4                 jz      short loc_4012FC
.text:004012F6                 cmp     esi, [esp+0Ch+lpFileName]
.text:004012FA                 jz      short loc_401305
.text:004012FC
.text:004012FC loc_4012FC:                             ; CODE XREF: sub_401290+64↑j
.text:004012FC                 push    edi             ; hMem
.text:004012FD                 call    ds:LocalFree
.text:00401303                 xor     edi, edi
.text:00401305
.text:00401305 loc_401305:                             ; CODE XREF: sub_401290+39↑j
.text:00401305                                         ; sub_401290+3E↑j ...
.text:00401305                 push    ebx             ; hObject
.text:00401306                 call    ds:CloseHandle
.text:0040130C                 mov     eax, [esp+0Ch+arg_4]
.text:00401310                 test    eax, eax
.text:00401312                 jz      short loc_401316
.text:00401314                 mov     [eax], esi
.text:00401316
.text:00401316 loc_401316:                             ; CODE XREF: sub_401290+82↑j
.text:00401316                 mov     eax, edi
```

If not another argument is used as a pointer to the result. and the `result is esi` **. In our case esi is the file size.**

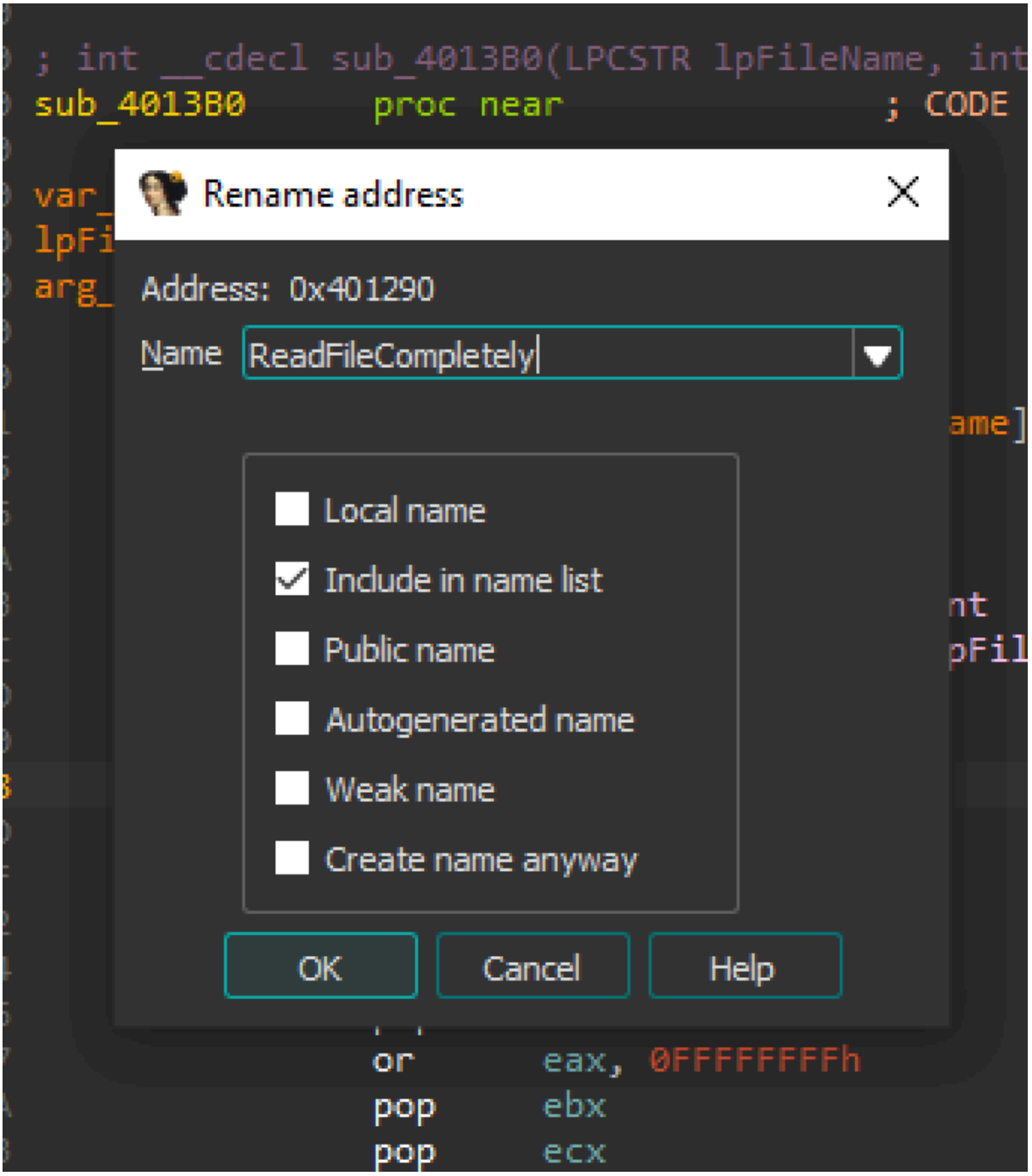
```
.text:00401303                 xor     edi, edi
.text:00401305
.text:00401305 loc_401305:                             ; CODE XREF: sub_401290+39↑j
.text:00401305                                         ; sub_401290+3E↑j ...
.text:00401305                 push    ebx             ; hObject
.text:00401306                 call    ds:CloseHandle
.text:0040130C                 mov     eax, [esp+0Ch+arg_4]
.text:00401310                 test    eax, eax
.text:00401312                 jz      short loc_401316
.text:00401314                 mov     [eax], esi
```

EAX, the standard return of the function is EDI that is the result of allocation of the memory.

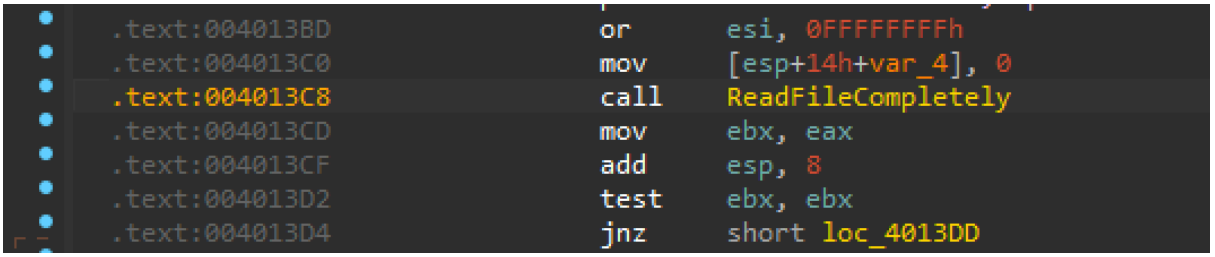
```
.text:00401312                 jz      short loc_401316
.text:00401314                 mov     [eax], esi
.text:00401316
.text:00401316 loc_401316:                             ; CODE XREF: sub_401290+82↑j
.text:00401316                 mov     eax, edi
.text:00401318                 pop     edi
.text:00401319                 pop     esi
.text:0040131A                 pop     ebx
.text:0040131B                 retn
```

So, the function is reading the the whole file. The file size is passed as a result in the pointer that is the second argument. And the buffer just returned in EAX.

Going back and renaming this function to something like ReadFile.



This function reading the contents of the gpca.dat



And it's address is `0x401290`

```
.text:00401290 arg_4           = dword ptr  8
.text:00401290
.text:00401290                 mov     eax, [esp+lpFileName]
.text:00401294                 push    ebx
.text:00401295                 push    0               ; hTemplateFile
.text:00401297                 push    80h             ; dwFlagsAndAttributes
.text:0040129C                 push    3               ; dwCreationDisposition
.text:0040129E                 push    0               ; lpSecurityAttributes
.text:004012A0                 push    0               ; dwShareMode
.text:004012A2                 push    80000000h       ; dwDesiredAccess
.text:004012A7                 push    eax             ; lpFileName
.text:004012A8                 call    ds:CreateFileA
```

Now we know where the content's of the file is read.

go one level up

we need to track the value of the file to find the location where it is decoded or decrypted.

`NOTE: - The result, the contents of the file is returned in EAX.` **Which is then immediately copied in in EBX.**

```
.text:004013BA                 push    esi
.text:004013BB                 push    eax             ; int
.text:004013BC                 push    ecx             ; lpFileName
.text:004013BD                 or      esi, 0FFFFFFFFh
.text:004013C0                 mov     [esp+14h+var_4], 0
.text:004013C8                 call    ReadFileCompletely
.text:004013CD                 mov     ebx, eax
.text:004013CF                 add     esp, 8
.text:004013D2                 test    ebx, ebx
.text:004013D4                 jnz     short loc_4013DD
.text:004013D6                 pop     esi
.text:004013D7                 or      eax, 0FFFFFFFFh
.text:004013DA                 pop     ebx
.text:004013DB                 pop     ecx
.text:004013DC                 retn
.text:004013DD ; ---------------------------------------------------------------------------
```

let's track EBX

It is used as a argument to another function.

```
.text:004013E6                 cmp     ecx, eax
.text:004013E8                 jb      short loc_401424
.text:004013EA                 push    eax
.text:004013EB                 push    ebx
.text:004013EC                 push    10h
.text:004013EE                 push    offset unk_405010
.text:004013F3                 mov     [esp+1Ch+var_4], eax
.text:004013F7                 call    sub_4032F0
.text:004013FC                 mov     eax, [ebx]
.text:004013FE                 add     esp, 10h
.text:00401401                 cmp     eax, 0A0B0C0D0h
```

`before calling the function the code is verifying. if var_4 is exactly 0x8438`

```
.text:004013DD
.text:004013DD loc_4013DD:                             ; CODE XREF: sub_4013B0+24↑j
.text:004013DD                 mov     ecx, [esp+0Ch+var_4]
.text:004013E1                 mov     eax, 8438h
.text:004013E6                 cmp     ecx, eax
.text:004013E8                 jb      short loc_401424
.text:004013EA                 push    eax
.text:004013EB                 push    ebx
.text:004013EC                 push    10h
.text:004013EE                 push    offset unk_405010
.text:004013F3                 mov     [esp+1Ch+var_4], eax
```

`var_4 is used as a second argument for ReadFileCompletely. So it is the size of the file.`

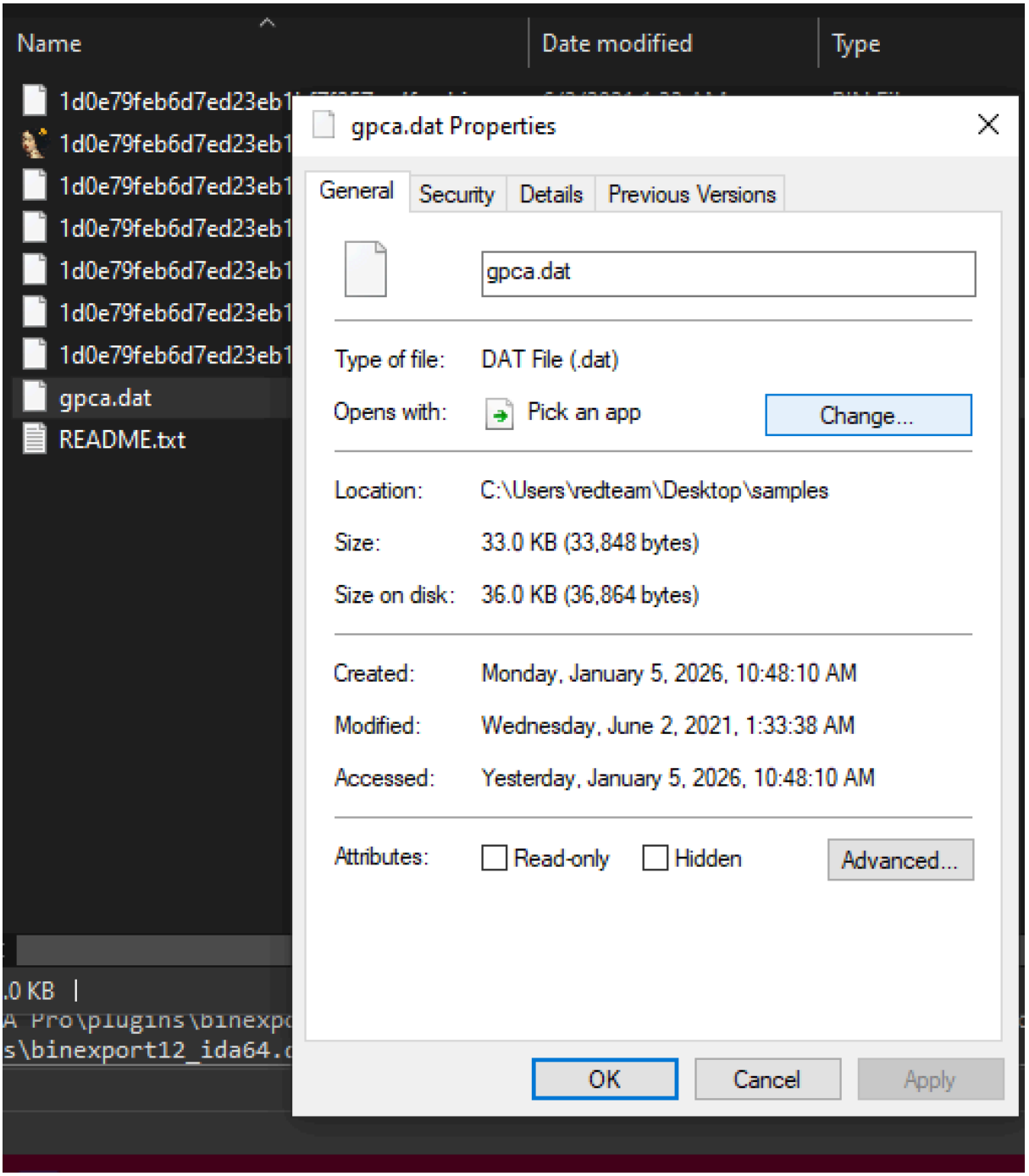
there's a file size check.

```
.text:004013B6                 lea     eax, [esp+8+var_4]
.text:004013BA                 push    esi
.text:004013BB                 push    eax             ; int
.text:004013BC                 push    ecx             ; lpFileName
.text:004013BD                 or      esi, 0FFFFFFFFh
.text:004013C0                 mov     [esp+14h+var_4], 0
.text:004013C8                 call    ReadFileCompletely
.text:004013CD                 mov     ebx, eax
.text:004013CF                 add     esp, 8
.text:004013D2                 test    ebx, ebx
.text:004013D4                 jnz     short loc_4013DD
.text:004013D6                 pop     esi
.text:004013D7                 or      eax, 0FFFFFFFFh
.text:004013DA                 pop     ebx
.text:004013DB                 pop     ecx
.text:004013DC                 retn
```

Convert the value of `0x8438` **to decimal. highlight the value and press** `h` **to convert hexadecimal to decimal. we can see it's 33848 bytes.**

```
.text:004013DD ; ---------------------------------------------------------------------------
.text:004013DD
.text:004013DD loc_4013DD:                             ; CODE XREF: sub_4013B0+24↑j
.text:004013DD                 mov     ecx, [esp+0Ch+var_4]
.text:004013E1                 mov     eax, 33848
.text:004013E6                 cmp     ecx, eax
.text:004013E8                 jb      short loc_401424
.text:004013EA                 push    eax
```

`Verifying with the file size of gpca.dat it's 33,848 bytes.` **So we are in the right path.**



we have another function call `sub_4032F0`

```
.text:004013EA    push    eax
.text:004013EB    push    ebx
.text:004013EC    push    10h
.text:004013EE    push    offset unk_405010
.text:004013F3    mov     [esp+1Ch+var_4], eax
.text:004013F7    call    sub_4032F0
.text:004013FC    mov     eax, [ebx]
```

that takes some pointer to unknown value. 16 or (0x10), ebx that is our encrypted file. and eax that is the size of the file. So it's look like it has to operate on this buffer somehow. so let's go into that function.

```
.text:004013EA                  push    eax
.text:004013EB                  push    ebx
.text:004013EC                  push    10h
.text:004013EE                  push    offset unk_405010
.text:004013F3                  mov     [esp+1Ch+var_4], eax
.text:004013F7                  call    sub_4032F0
.text:004013FC                  mov     eax, [ebx]
```

Function: `sub_4032F0`

```
.text:004032E8                  retn
.text:004032E8 sub_403250       endp
.text:004032E8
.text:004032E8 ; ---------------------------------------------------------------------------
.text:004032E9                  align 10h
.text:004032F0
.text:004032F0 ; =============== S U B R O U T I N E =======================================
.text:004032F0
.text:004032F0
.text:004032F0 sub_4032F0       proc near               ; CODE XREF: sub_4013B0+47↑p
.text:004032F0
.text:004032F0 var_104          = byte ptr -104h
.text:004032F0 var_103          = byte ptr -103h
.text:004032F0 arg_0            = dword ptr  4
.text:004032F0 arg_4            = dword ptr  8
.text:004032F0 arg_8            = dword ptr  0Ch
.text:004032F0 arg_C            = dword ptr  10h
.text:004032F0
.text:004032F0                  sub     esp, 104h
```

let's rename two last arguments to from:

arg_8 → `argGpcaBuffer`

arg_c → `argGpcaLen` **length of the buffer.**

NOTE: arguments load in the memory in the reverse order.

```
.text:004032F0
.text:004032F0 sub_4032F0       proc near               ; CODE XREF: sub_4013B0+47↑p
.text:004032F0
.text:004032F0 var_104          = byte ptr -104h
.text:004032F0 var_103          = byte ptr -103h
.text:004032F0 arg_0            = dword ptr  4
.text:004032F0 arg_4            = dword ptr  8
.text:004032F0 argGpcaBuffer    = dword ptr  0Ch
.text:004032F0 argGpcaLen       = dword ptr  10h
.text:004032F0
```

These arguments are used in another function call.

```
.text:0040331D                push    ecx
.text:0040331E                push    edx
.text:0040331F                call    sub_4031E0
.text:00403324                mov     eax, [esp+114h+argGpcaLen]
.text:0040332B                lea     ecx, [esp+114h+var_104]
.text:0040332F                push    eax
.text:00403330                mov     eax, [esp+118h+argGpcaBuffer]
.text:00403337                push    eax
.text:00403338                push    eax
.text:00403339                push    ecx
.text:0040333A                call    sub_403250
```

So, these are used in another function call. But that one is the second in the function.

we have one function with two different calls. and first one is using only the first and the second argument that we have not recognized yet.

```
.text:004032FC                xor     eax, eax
.text:004032FE                lea     edi, [esp+108h+var_103]
.text:00403302                mov     [esp+108h+var_104], 0
.text:00403307                lea     edx, [esp+108h+var_104]
.text:0040330B                rep stosd
.text:0040330D                mov     ecx, [esp+108h+arg_0]
.text:00403314                stosb
.text:00403315                mov     eax, [esp+108h+arg_4]
.text:0040331C                push    eax
.text:0040331D                push    ecx
.text:0040331E                push    edx
.text:0040331F                call    sub_4031E0
.text:00403324                mov     eax, [esp+114h+argGpcaLen]
.text:0040332B                lea     ecx, [esp+114h+var_104]
.text:0040332F                push    eax
.text:00403330                mov     eax, [esp+118h+argGpcaBuffer]
.text:00403337                push    eax
.text:00403338                push    eax
```

Analyzing the first function.

```
.text:0040331D                push    ecx
.text:0040331E                push    edx
.text:0040331F                call    sub_4031E0
.text:00403324                mov     eax, [esp+114h+argGpcaLen]
.text:0040332B                lea     ecx, [esp+114h+var_104]
.text:0040332F                push    eax
.text:00403330                mov     eax, [esp+118h+argGpcaBuffer]
.text:00403337                push    eax
.text:00403338                push    eax
.text:00403339                push    ecx
.text:0040333A                call    sub_403250
```

Function: `0x4031E0`

we have some loop for 256 iterations filling a buffer.

```
.text:004031E0 arg_0           = dword ptr  4
.text:004031E0 arg_4           = dword ptr  8
.text:004031E0 arg_8           = dword ptr  0Ch
.text:004031E0
.text:004031E0                 push    ebx
.text:004031E1                 push    ebp
.text:004031E2                 push    esi
.text:004031E3                 mov     esi, [esp+0Ch+arg_0]
.text:004031E7                 xor     ecx, ecx
.text:004031E9                 push    edi
.text:004031EA                 xor     eax, eax
.text:004031EC
.text:004031EC loc_4031EC:                             ; CODE XREF: sub_4031E0+15↓j
.text:004031EC                 mov     [eax+esi], al
.text:004031EF                 inc     eax
.text:004031F0                 cmp     eax, 100h
.text:004031F5                 jl      short loc_4031EC
.text:004031F7                 mov     edi, [esp+10h+arg_8]
.text:004031FB                 mov     ebp, [esp+10h+arg_4]
.text:004031FF                 mov     [esi+100h], cl
.text:00403205                 mov     [esi+101h], cl
.text:0040320B                 mov     byte ptr [esp+10h+arg_0], cl
.text:0040320F
.text:0040320F loc_40320F:                             ; CODE XREF: sub_4031E0+60↓j
.text:0040320F                 mov     eax, ecx
```

Another loop that is operating on the buffer. nothing looks like encryption.

```
.text:0040320F
.text:0040320F loc_40320F:                             ; CODE XREF: sub_4031E0+60↓j
.text:0040320F                 mov     eax, ecx
.text:00403211                 mov     bl, [ecx+esi]
.text:00403214                 cdq
.text:00403215                 idiv    edi
.text:00403217                 mov     al, [edx+ebp]
.text:0040321A                 mov     dl, byte ptr [esp+10h+arg_0]
.text:0040321E                 add     al, bl
.text:00403220                 add     dl, al
.text:00403222                 mov     byte ptr [esp+10h+arg_0], dl
.text:00403226                 mov     eax, [esp+10h+arg_0]
.text:0040322A                 and     eax, 0FFh
.text:0040322F                 add     eax, esi
.text:00403231                 inc     ecx
.text:00403232                 cmp     ecx, 100h
.text:00403238                 mov     dl, [eax]
.text:0040323A                 mov     [ecx+esi-1], dl
.text:0040323E                 mov     [eax], bl
.text:00403240                 jl      short loc_40320F
```

Analysing another

function: `sub_403250`

This one also operates in a loop using index 0x100 and 0x101.

```
.text:00403272
.text:00403272 loc_403272:                             ; CODE XREF: sub_403250+92↓j
.text:00403272                 mov     bl, [eax+100h]
.text:00403278                 inc     bl
.text:0040327A                 mov     cl, bl
.text:0040327C                 mov     [eax+100h], bl
.text:00403282                 mov     bl, [eax+101h]
.text:00403288                 and     ecx, 0FFh
.text:0040328E                 lea     edx, [ecx+eax]
.text:00403291                 mov     cl, [ecx+eax]
.text:00403294                 add     bl, cl
.text:00403296                 mov     cl, bl
.text:00403298                 mov     [eax+101h], bl
.text:0040329E                 and     ecx, 0FFh
.text:004032A4                 add     ecx, eax
.text:004032A6                 mov     edi, ecx
.text:004032A8                 mov     cl, [edx]
.text:004032AA                 mov     bl, [edi]
.text:004032AC                 mov     [edx], bl
.text:004032AE                 mov     [edi], cl
.text:004032B0                 xor     edx, edx
.text:004032B2                 xor     ecx, ecx
.text:004032B4                 mov     dl, [eax+101h]
.text:004032BA                 mov     cl, [eax+100h]
.text:004032C0                 mov     dl, [edx+eax]
```

We have a xor loop that is xoring a buffer.

there's a xor cl, bl → that is not just cleaning a register but actually xoring two different values from buffers.

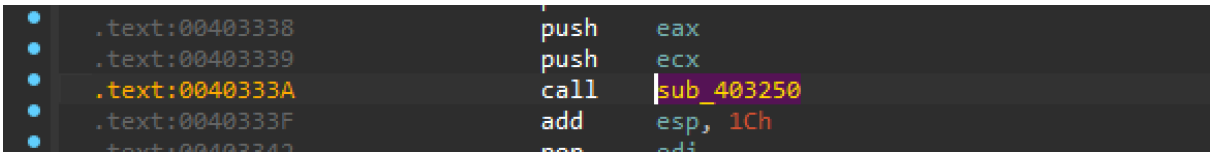
```
.text:004032AE                 mov     [edi], cl
.text:004032B0                 xor     edx, edx
.text:004032B2                 xor     ecx, ecx
.text:004032B4                 mov     dl, [eax+101h]
.text:004032BA                 mov     cl, [eax+100h]
.text:004032C0                 mov     dl, [edx+eax]
.text:004032C3                 mov     bl, [ecx+eax]
.text:004032C6                 add     dl, bl
.text:004032C8                 mov     bl, [esi+ebp]
.text:004032CB                 and     edx, 0FFh
.text:004032D1                 mov     cl, [edx+eax]
.text:004032D4                 xor     cl, bl
.text:004032D6                 mov     [esi], cl
.text:004032D8                 mov     ecx, [esp+10h+arg_C]
.text:004032DC                 inc     esi
.text:004032DD                 dec     ecx
.text:004032DE                 mov     [esp+10h+arg_C], ecx
.text:004032E2                 jnz     short loc_403272
.text:004032E4                 pop     edi
.text:004032E5                 pop     esi
.text:004032E6                 pop     ebp
.text:004032E7                 pop     ebx
```

The destination of the xor loop is `esi, [esp+0Ch+arg_8]. In our case it will be our GpcaBuffer` .

```
.text:00403261                 push    ebp
.text:00403262                 mov     ebp, [esp+8+arg_4]
.text:00403266                 push    esi
.text:00403267                 mov     esi, [esp+0Ch+arg_8]
.text:0040326B                 push    edi
.text:0040326C                 sub     ebp, esi
```

So the function `sub_403250` **is definitely xoring somehow our input buffer.**

It's look like it is decryption.



we need to figure out which algorithm.

We will use templates (source code and combine it with the disassembly that we have in the IDA to recognize) the algorithm.

Template for Rc4

```
RC4 (ARC4)
/*
 * ARC4 key schedule
 */
void mbedtls_arc4_setup( mbedtls_arc4_context *ctx, const unsigned char
*key,
                         unsigned int keylen )
{
    int i, j, a;
    unsigned int k;
    unsigned char *m;
    ctx->x = 0;
    ctx->y = 0;
    m = ctx->m;
    for( i = 0; i < 256; i++ )
        m[i] = (unsigned char) i;
    j = k = 0;
    for( i = 0; i < 256; i++, k++ )
    {
        if( k >= keylen ) k = 0;
        a = m[i];
        j = ( j + a + key[k] ) & 0xFF;
        m[i] = m[j];
        m[j] = (unsigned char) a;
```

```
    }
}
```

```
.text:0040331D                push    ecx
.text:0040331E                push    edx
.text:0040331F                call    RC4SetKey
.text:00403324                mov     eax, [esp+114h+argGpcaLen]
.text:0040332B                lea     ecx, [esp+114h+var_104]
.text:0040332F                push    eax
.text:00403330                mov     eax, [esp+118h+argGpcaBuffer]
```

Renaming `sub_403250` → **RC4**

RC4 cipher needs a key and key length. We need to track the key and the length.

Function: RC4SetKey

```
.text:004031E0
.text:004031E0 RC4SetKey      proc near               ; CODE XREF: RC4+2F↓p
.text:004031E0
.text:004031E0 arg_0          = dword ptr  4
.text:004031E0 arg_4          = dword ptr  8
.text:004031E0 arg_8          = dword ptr  0Ch
.text:004031E0
.text:004031E0                push    ebx
```

arg_0 is a S-box context

renaming arg_0 → argCtx

We have 2 arguments arg_4 and arg_8.

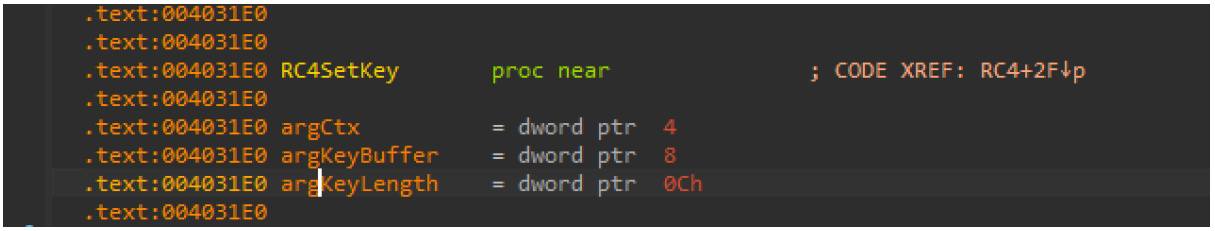
```
.text:004031E0
.text:004031E0 RC4SetKey      proc near               ; CODE XREF: RC4+2F↓p
.text:004031E0
.text:004031E0 argCtx         = dword ptr  4
.text:004031E0 arg_4          = dword ptr  8
.text:004031E0 arg_8          = dword ptr  0Ch
```

arg_4 is used as a buffer. So it is the key buffer.

arg_4 → argKeyBuffer

arg_4 is modulo, So this is the length.

arg4 → argkeyLength



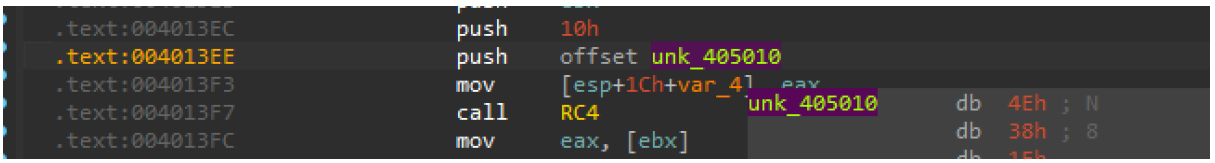
So we have 4 arguments.

KeyBuffer, KeyLength, argGpcsBuffer, argGpcaLen

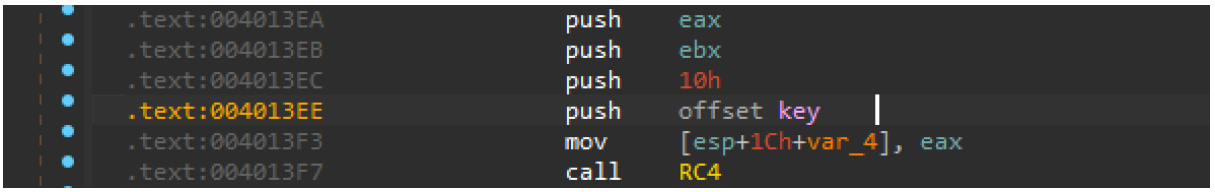
They go in reverse order.

The first argument is going to be a key

renaming it to key



Key length is 0x10 or 16 bytes.



EBX is our value of gpca.dat, and EAX - the length of that buffer.

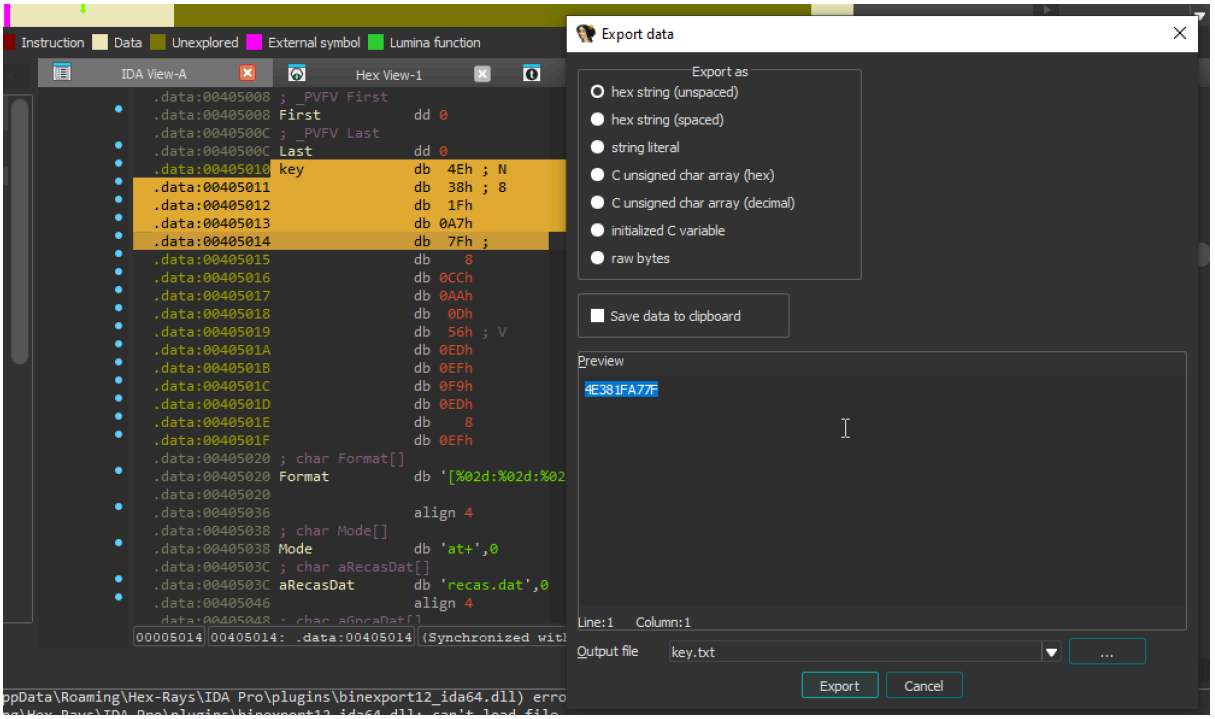
Template for Decryption

```
#!/usr/bin/python3
# - common decryption
```

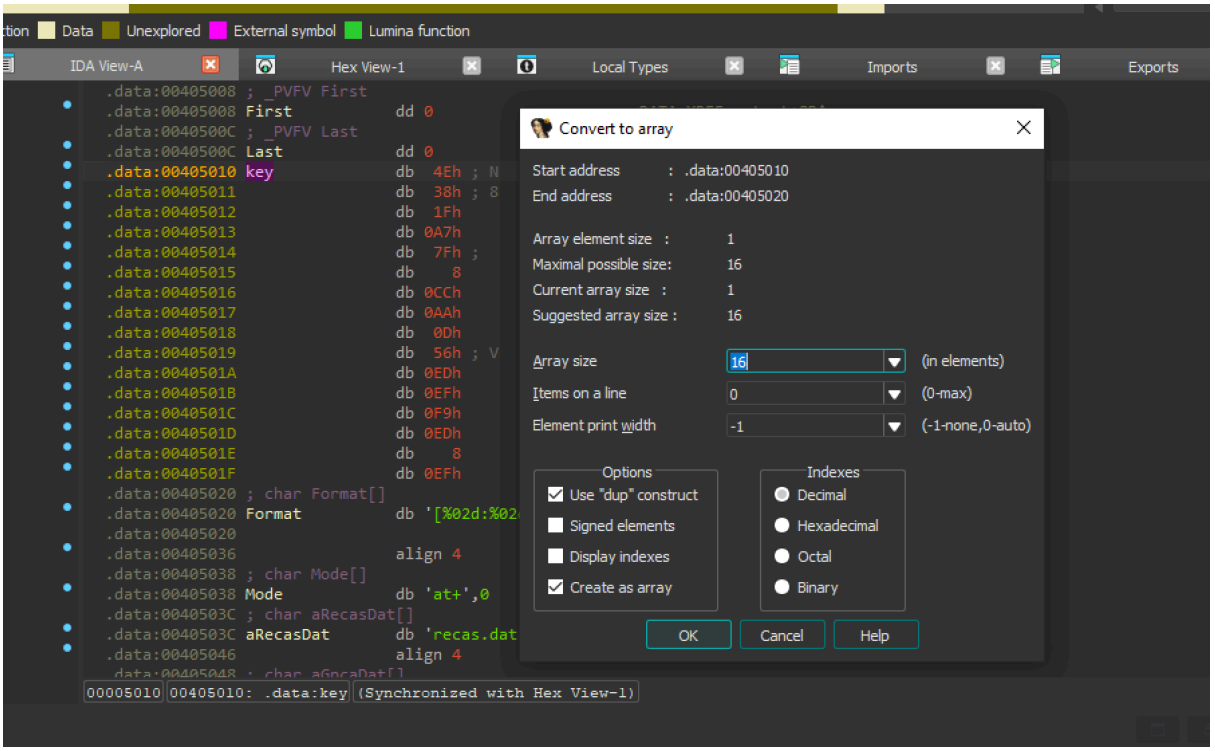
```
# - identifying the crypto algorithm
# - using a decoding framework

from decoder_core import *
from Crypto.Cipher import AES, Salsa20, ARC2, ARC4, Blowfish, DES3, DE
S

class Decoder(TrainingDecoder):
    def __init__(self):
        # MD5 hash of the correctly decrypted file
        super().__init__()

    def decode(self):
        # self.data is a bytearray with the contents of the file

        # 1. Insert the key as a hexadecimal string:
        key = bytes.fromhex(REPLACE_ME_HERE)

        # 2. Identify the decryption algorithm and use it
        # examples:
        # cipher = AES.new(key, AES.MODE_EAX)
        # cipher = ARC4.new(key)
        # cipher = Salsa20.new(key=key)
        # cipher = ARC2.new(key, ARC2.MODE_CFB)
        # cipher = Blowfish.new(key, Blowfish.MODE_CBC)
        # cipher = DES3.new(key, DES3.MODE_CFB)
        # cipher = DES.new(key, DES.MODE_OFB)
        cipher = REPLACE_ME_HERE # ← Create a proper decryption class h
ere

        # If you use the correct algorithm and the key, the decrypted data will
be written
        # in a .dec file
        self.data = cipher.decrypt(bytes(self.data))

        # Check if the results are correct
        self.check_results()
```

```
# Create the decryptor object. Automation happens in __init__()
Decoder()
```

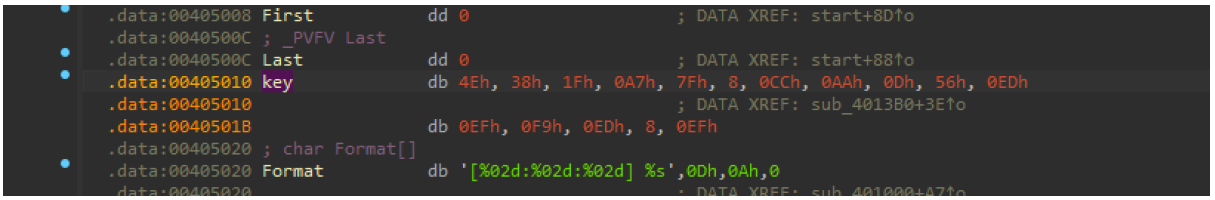
Extraction of the key.



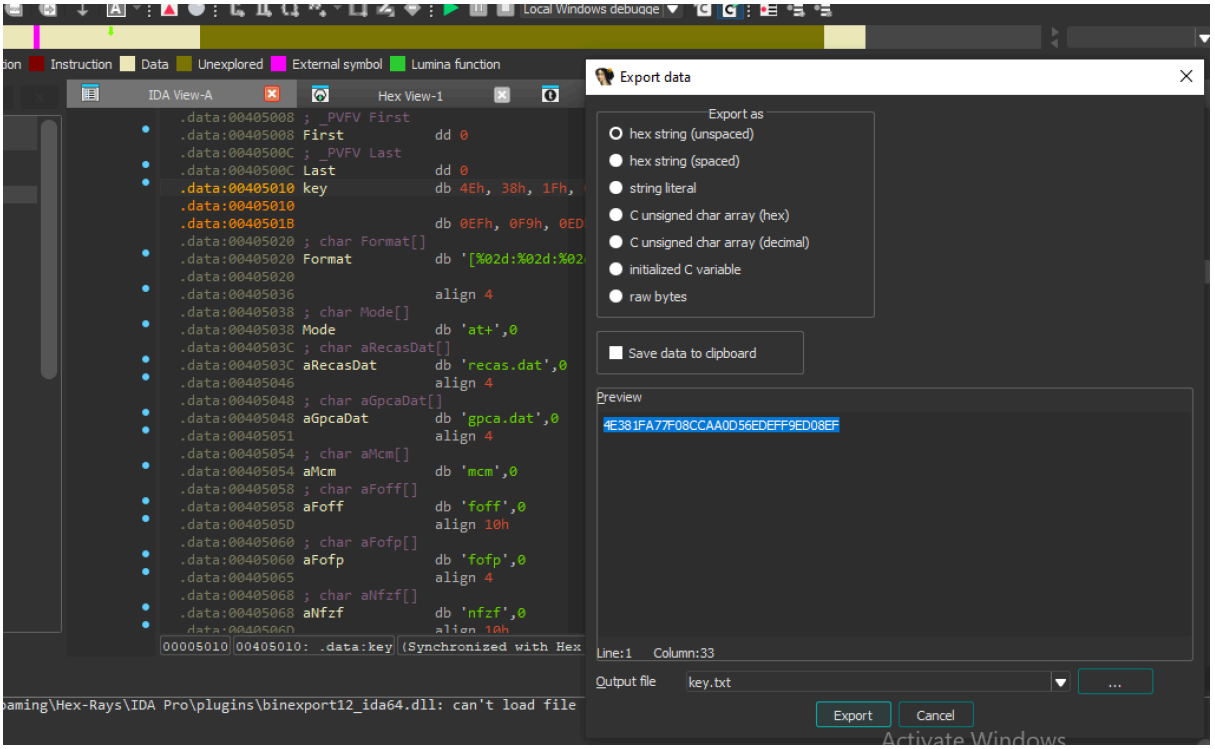
Or we can create an array. Ida automatically recognize the boundaries.



press ok there will be now an array. copy the key.



Extract data hex string



key:

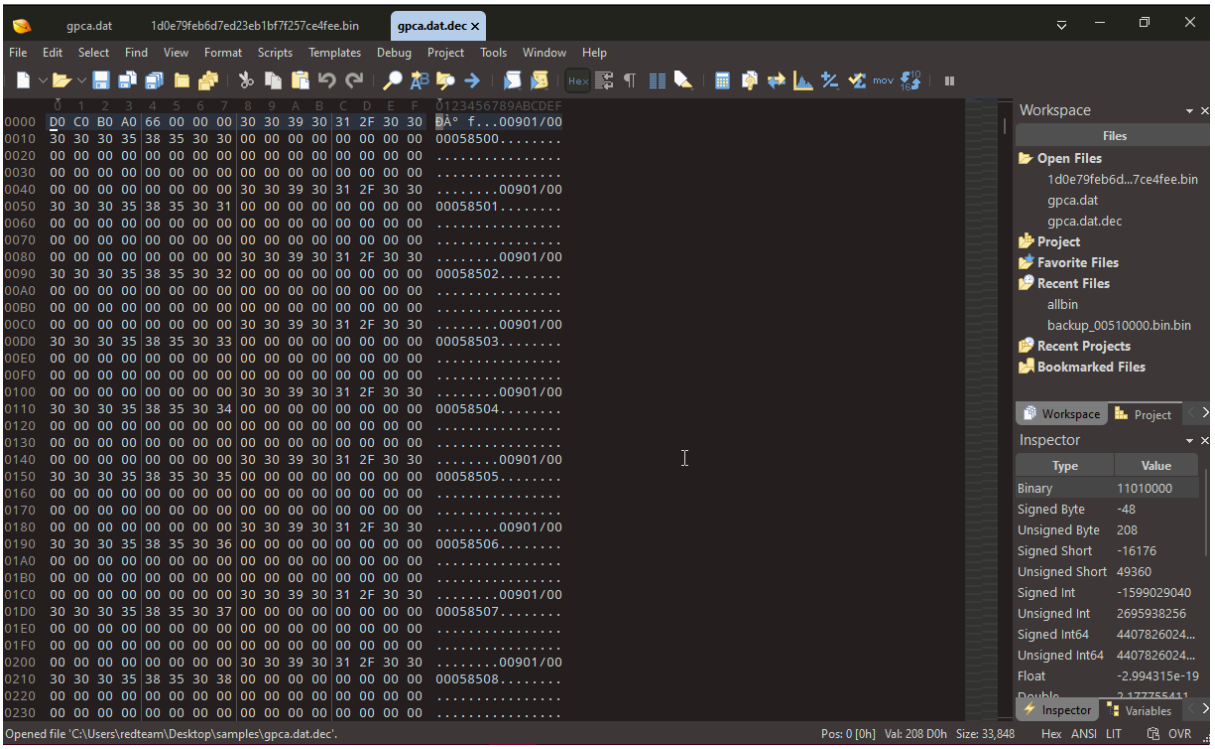
```
4E381FA77F08CCAA0D56EDEFF9ED08EF
```

Decryption:

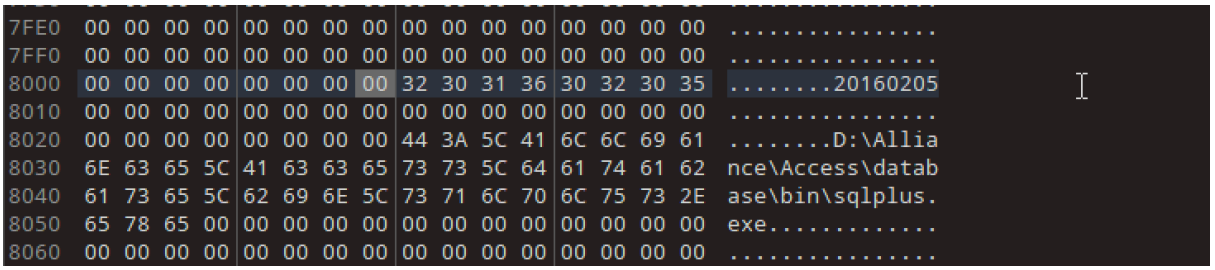
```
kant@APPLEs-MacBook-Pro ~/D/gpca> python3 task04_gpca_dat.py gpca.dat   (cyber)
Raw loaded
The MD5 checksum of the decoded file is:
b269f00944fca6f74733e0f7232728dc
Written the results to gpca.dat.dec
kant@APPLEs-MacBook-Pro ~/D/gpca>                                       (cyber)
```

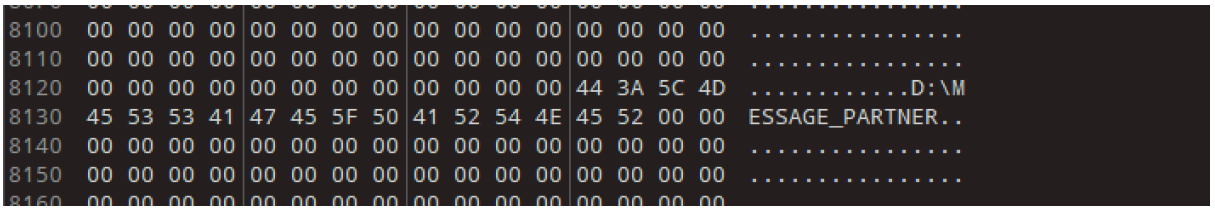
Opening the decrypted file in hex editor.

We have a readable strings and these were swift transactions.

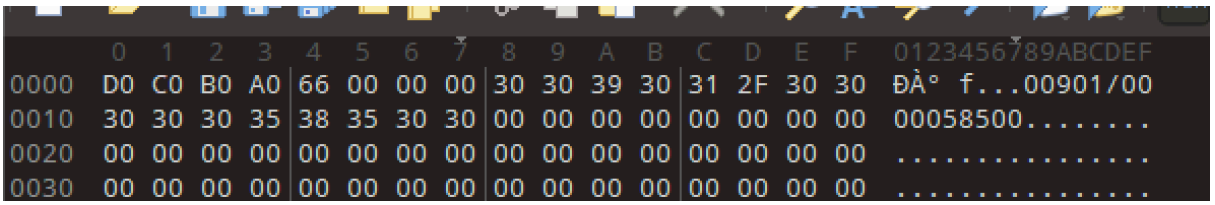


Some more Interesting strings.





First bytes are `D0 C0 B0 A0`



```
        0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
0000   D0 C0 B0 A0 66 00 00 00 30 30 39 30 31 2F 30 30  ÐÀ° f...00901/00
0010   30 30 30 35 38 35 30 30 00 00 00 00 00 00 00 00  00058500........
0020   00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0030   00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

If we go back to IDA.

After calling RC4. The code reads the first DWORD of the decrypted buffer at address `004013FC` and it verifies if there is a particular magic value. that is exactly `D0 C0 B0 A0` or just if you rotate in little-endian.

This assures we have Decrypted the whole file correctly.

```
.text:004013DB                pop     ecx
.text:004013DC                retn
.text:004013DD ; ---------------------------------------------------------------------------
.text:004013DD
.text:004013DD loc_4013DD:                            ; CODE XREF: sub_4013B0+24↑j
.text:004013DD                mov     ecx, [esp+0Ch+var_4]
.text:004013E1                mov     eax, 33848
.text:004013E6                cmp     ecx, eax
.text:004013E8                jb      short loc_401424
.text:004013EA                push    eax
.text:004013EB                push    ebx
.text:004013EC                push    10h
.text:004013EE                push    offset key
.text:004013F3                mov     [esp+1Ch+var_4], eax
.text:004013F7                call    RC4
.text:004013FC                mov     eax, [ebx]
.text:004013FE                add     esp, 10h
.text:00401401                cmp     eax, 0A0B0C0D0h
.text:00401406                jnz     short loc_401424
.text:00401408                mov     ecx, [esp+0Ch+var_4]
.text:0040140C                push    edi
.text:0040140D                mov     edi, [esp+10h+arg_4]
.text:00401411                mov     edx, ecx
.text:00401413                mov     esi, ebx
.text:00401415                shr     ecx, 2
.text:00401418                rep movsd
.text:0040141A                mov     ecx, edx
.text:0040141C                and     ecx, 3
.text:0040141F                rep movsb
.text:00401421                xor     esi, esi
.text:00401423                pop     edi
.text:00401424
```

Decryption Program:

```
#!/usr/bin/python3
```

```
from decoder_core import *
from Crypto.Cipher import AES, Salsa20, ARC2, ARC4, Blowfish, DES3, DE
S

class Decoder(TrainingDecoder):
    def __init__(self):
        # MD5 hash of the correctly decrypted file
        super().__init__()

    def decode(self):
        # self.data is a bytearray with the contents of the file

        # 1. Insert the key as a hexadecimal string:
        key = bytes.fromhex(REPLACE_ME)

        # 2. Identify the decryption algorithm and use it
        # examples:
        # cipher = AES.new(key, AES.MODE_EAX)
        # cipher = ARC4.new(key)
        # cipher = Salsa20.new(key=key)
        # cipher = ARC2.new(key, ARC2.MODE_CFB)
        # cipher = Blowfish.new(key, Blowfish.MODE_CBC)
        # cipher = DES3.new(key, DES3.MODE_CFB)
        # cipher = DES.new(key, DES.MODE_OFB)
        # cipher = ARC4.new(key) # <-- Create a proper decryption class here

        # If you use the correct algorithm and the key, the decrypted data will
be written
        # in a .dec file
        self.data = cipher.decrypt(bytes(self.data))

        # Check if the results are correct
        self.check_results()

# Create the decryptor object. Automation happens in __init__()
Decoder()
```